

Fase 1: Provider

Objetivo

Validar de forma exhaustiva que un participante provider puede, vía API:

- autenticarse;
- crear y mantener vocabularios;
- crear y mantener policy definitions;
- crear y mantener assets;
- construir contract definitions reutilizando políticas existentes;
- y dejar el recurso preparado para su exposición y consumo.

Alcance

Esta fase cubre **solo provider**.

No entra todavía en negociación consumer ni en transferencia, salvo la comprobación mínima de que lo publicado queda accesible para la siguiente fase.

Qué entra y qué no entra

Aquí conviene separar dos niveles:

1. Gobernanza del dataspace

El Registration Service sirve para registrar participantes y conectores dentro del espacio de datos. Eso es útil como precondition o comprobación de contexto, pero no es el núcleo del flujo provider de publicación de recursos.

2. Gestión real del provider

El trabajo fuerte del provider ocurre en el **Management API** del conector: assets, vocabularios, políticas, contract definitions, catálogo, contadores y demás.



AVISO: tratar **Registration Service** como **precheck opcional**, y centrar la validación en el **Management API del conector**.

Arquitectura mínima del flujo provider

Precondición 1: autenticación

La documentación describe un flujo de login contra **Keycloak**, realm `dataspace`, client `dataspace-users`, y uso posterior del JWT en `Authorization: Bearer <token>` para Management API y otros endpoints protegidos.

Precondición 2: conector operativo

Como el conector es el core, si está levantado se puede operar todo por API aunque no haya UI. Si el conector no responde, no hay provider real que validar.

Precondición 3: participante registrado, si aplica

Si la validación se hace en un entorno de dataspace real, puede tener sentido comprobar que el conector está dado de alta en Registration Service mediante `/participants` o `/public/participants`. Esta comprobación debe tratarse como contexto, no como flujo principal de publicación.

Bloques de validación provider por API

Bloque 0. Prechecks

Antes de entrar en CRUD funcional, conviene hacer tres comprobaciones rápidas:

0.1. Login

Obtener JWT desde Keycloak y verificar que sirve para llamadas protegidas.

0.2. Registro del participante

Opcionalmente, comprobar que el conector aparece en Registration Service:

- `GET /participants`
- `GET /public/participants`

0.3. Dataplane

No es estrictamente el primer paso funcional del provider, pero sí conviene verificar si el dataplane está ya registrado, porque después afectará a consumo y transferencia. Existe `POST /management/v2/dataplanes` para registrar y `GET /management/v2/dataplanes` para listar.

Bloque 1. Vocabularios

Los vocabularios definen **esquemas de metadatos** y **categorías de assets**, y se usan para construir formularios y validar metadatos en las UIs. A nivel API, el conector expone operaciones para crear, actualizar, listar, obtener y borrar vocabularios.

Endpoints base

- `PUT /management/vocabularies`
- `POST /management/vocabularies/request`
- `GET /management/vocabularies/{id}`
- `DELETE /management/vocabularies/{id}`

Qué validar

Crear

- vocabulario mínimo válido;
- vocabulario con categoría `dataset`;
- vocabulario con otra categoría si aplica.

Leer

- recuperar por id;
- listar;
- comprobar persistencia tras recarga de sesión.

Editar

- cambiar `name`;
- cambiar `category`;
- cambiar `jsonSchema`.

Borrar

- borrar uno no usado;
- comprobar qué ocurre al intentar borrar uno referenciado indirectamente por activos ya modelados.



IMPORTANTE: Esto último no está explicitado en la documentación como restricción formal, así que aquí conviene tratarlo como hipótesis de prueba, no como comportamiento garantizado. La parte documentada es que sí existe operación de borrado por id.

Validación negativa

- `jsonSchema` mal formado;
- `category` inválida;
- payload incompleto;
- ids duplicados.

La documentación no detalla todos los errores esperables, así que aquí el objetivo es descubrir cómo responde realmente la API.

Caso base recomendado

Usar como payload un vocabulario tipo `generic-dataset-vocabulary.json`, con `jsonSchema` escapado y campos básicos como `name`, `description` y `keywords`.

Bloque 2. Policy Definitions

El conector expone `policydefinitions` en v3, además de endpoints avanzados para complex policies y, en v3.1-alpha, validación y evaluation plan. A nivel funcional del provider, lo importante es que las políticas se crean por separado y luego se reutilizan en los contratos.

Endpoints base

- `POST /management/v3/policydefinitions`
- `POST /management/v3/policydefinitions/request`
- `GET /management/v3/policydefinitions/{id}`

- `PUT /management/v3/policydefinitions/{id}`
- `DELETE /management/v3/policydefinitions/{id}`

Endpoints avanzados

- `POST /v3/complexpolicydefinitions`
- `POST /v3.1alpha/policydefinitions/{id}/evaluationplan`
- `POST /v3.1alpha/policydefinitions/{id}/validate`

Qué validar

Crear Access Policy

- caso simple con restricción `PURPOSE = research`.

Crear Contract Policy

- caso simple con permiso `USE` y obligación `NOTIFY`.

Leer / listar / editar / borrar

- comprobar persistencia;
- comprobar si el contenido ODRL vuelve igual o sufre normalización;
- verificar si el id queda bien reutilizable después.

Validación negativa

- política vacía;
- ODRL JSON-LD mal formado;
- tipo o semántica incoherente;
- id duplicado.

Aquí, igual que antes, el objetivo es mapear el comportamiento real de la API. La documentación confirma existencia de operaciones y modelos, pero no enumera todos los errores.

Validación avanzada opcional

- usar `validate`;
- usar `evaluationplan`;
- comprobar si esas rutas ayudan a detectar errores antes de crear o usar una política.

Bloque 3. Assets

El Management API del conector soporta creación, actualización, consulta, borrado y listado de assets. Los tipos explícitos documentados son `dataset`, `service` y `machineLearning`. Además, existe soporte específico para carga S3 / InesDataStore.

Endpoints base

- `POST /management/v3/assets`
- `PUT /management/v3/assets`
- `POST /management/v3/assets/request`
- `GET /management/v3/assets/{id}`
- `DELETE /management/v3/assets/{id}`

Carga interna / S3

- `POST /management/s3assets`
- `POST /s3assets/upload-chunk`
- `POST /s3assets/finalize-upload`

Qué validar

Crear por tipo

- un `dataset` ;
- un `service` ;
- un `machineLearning` .

Crear por modo de almacenamiento

- asset referenciado;
- asset con `InesDataStore` ;
- flujo chunked si se requiere validar subida real de fichero.

CRUD

- crear;
- recuperar;

- listar con `QuerySpec` ;
- actualizar;
- borrar.

Metadatos

- metadatos mínimos;
- metadatos completos;
- metadatos alineados con el vocabulario creado;
- metadatos deliberadamente incorrectos para ver si la API valida o deja pasar inconsistencias.

La documentación sí deja claro que el vocabulario modela esquemas y categorías; lo que ya no queda garantizado aquí es cuánto de esa validación se hace en la API y cuánto se delega a capas superiores. Eso es precisamente algo muy bueno a comprobar.

Validación negativa

- asset sin `assetType` ;
- referencias rotas;
- payload incompleto;
- borrado de asset aún usado por un contract definition.

La existencia de la relación asset-contrato sí está documentada; la reacción exacta al borrado en cascada no. Eso debe medirse empíricamente.

Bloque 4. Contract Definitions

Aquí está el ensamblaje real del provider. La documentación lo deja bastante claro: un contrato se construye sobre **un asset** y reutiliza **una Access Policy** y **una Contract Policy** ya existentes. A nivel API, eso se expresa con

`accessPolicyId` , `contractPolicyId` y `assetsSelector` .

Endpoints base

- `POST /management/v3/contractdefinitions`
- `POST /management/v3/contractdefinitions/request`
- `GET /management/v3/contractdefinitions/{id}`

- `DELETE /management/v3/contractdefinitions/{id}`

Payload base documentado

La propia documentación da un ejemplo con:

- `@id`
- `accessPolicyId`
- `contractPolicyId`
- `assetsSelector` con `operandLeft` sobre el id del asset y `operator = in`.

Qué validar

Crear

- contract definition para un único asset;
- contract definition para varios assets;
- contract definition reutilizando las mismas políticas sobre distintos assets.

Leer / listar / borrar

- recuperar por id;
- listar;
- borrar y comprobar si desaparece de resultados posteriores.

Validación negativa

- sin `accessPolicyId`;
- sin `contractPolicyId`;
- selector a asset inexistente;
- selector mal formado;
- ids duplicados.

Qué se busca confirmar aquí

No solo si “crea”, sino si la relación modular queda bien montada:

- política correcta;
- asset correcto;
- oferta contractual reutilizable después por consumer.

Bloque 5. Verificación de publicación

Aunque la fase sigue en provider, esta parte es clave. Tras crear asset y contract definition, conviene comprobar que lo publicado **sale al catálogo**. La documentación distingue dos vistas: catálogo remoto directo contra otro conector y catálogo federado cacheado.

Endpoints útiles

- `POST /management/v3/catalog/request`
- `POST /federatedcatalog/v1alpha/catalog/query`
- `POST /management/federatedcatalog/request`

Qué validar

Catálogo remoto

- si otro conector puede ver la oferta apuntando a `counterPartyAddress` y `protocol`.

Catálogo federado

- si el asset / oferta aparece en el catálogo agregado.

Coherencia

- si el asset visible corresponde al publicado;
- si la oferta contractual cuadra con el contract definition creado.

La documentación deja claro que el catálogo y las ofertas son el puente hacia consumer; esta verificación es la salida natural del provider.

Bloque 6. Contadores y trazabilidad básica

El conector expone un endpoint genérico de conteo `POST /management/pagination/count?type=...`, útil para validar que el estado del sistema cambia después de cada operación. Soporta, entre otros, `asset`, `policyDefinition`, `contractDefinition`, `contractAgreement`, `transferProcess` y `federatedCatalog`.

Cómo usarlo

Después de cada alta o borrado:

- contar assets;
- contar policy definitions;
- contar contract definitions.

Eso permite una comprobación rápida de persistencia sin depender solo de listados completos.

Orden recomendado de ejecución

Se recomienda ejecutarlo en cinco rondas.

Ronda 1. Smoke test

- login;
- listar participants si se requiere contexto;
- listar dataplanes;
- crear un vocabulario mínimo;
- crear una policy mínima;
- crear un asset mínimo.

Ronda 2. CRUD completo

- vocabularios;
- políticas;
- assets;
- contract definitions.

Ronda 3. Casos negativos

- payloads inválidos;
- ids duplicados;
- referencias rotas;
- borrados conflictivos.

Ronda 4. Encadenamiento real

- vocabulario;
- access policy;
- contract policy;
- asset;
- contract definition;
- publicación en catálogo.

Ronda 5. Repetición cruzada

- repetir el mismo flujo en otro espacio o con otro conector;
- comparar si el comportamiento es estable o depende del despliegue.

La arquitectura documentada permite ese tipo de validación porque los conectores y sus APIs son la unidad funcional básica.

Qué sería “probar absolutamente todo” en provider por API

Se pueden usar estas siete preguntas para cada módulo:

Crear

¿acepta el alta?

Leer

¿se puede recuperar con claridad?

Listar / contar

¿aparece donde debería aparecer?

Editar

¿los cambios persisten?

Borrar

¿elimina correctamente?

Validar

¿rechaza payloads malos o deja pasar basura?

Propagar

¿lo creado se puede reutilizar en el siguiente eslabón del flujo?

Registro de incidencias recomendado

Para el registro de incidencias se recomienda una ficha muy corta:

Módulo

Auth / Participants / Dataplanes / Vocabularios / Políticas / Assets / Contract Definitions / Catálogo

Caso de prueba

nombre corto

Precondición

qué debía existir

Endpoint

ruta exacta

Payload

versión resumida o referencia

Esperado

qué debía pasar

Observado

qué pasó

Clasificación

bug / limitación / integración / validación insuficiente

Severidad

bloqueante / alta / media / baja

IDs generados

vocabulario / policy / asset / contract definition

Evidencia

respuesta HTTP, captura, log

Recomendación práctica

Se recomienda empezar con **un caso base muy controlado**:

1. login;
2. crear un vocabulario dataset;
3. crear una access policy;
4. crear una contract policy;
5. crear un asset dataset;
6. crear un contract definition para ese asset;
7. verificar aparición en catálogo.

Después se repetiría exactamente el mismo guion con:

- un `service`,
 - un `machineLearning`,
 - y, si se requiere ampliar la validación, un asset subido vía `InesDataStore`.
-

Comandos del taller

Esta sección contiene los `curl` exactos que se ejecutan en directo durante la sesión. Asume que ya se han exportado las variables de entorno descritas en la Taller APIs - Plan de ejecución.

- `dataspace`
- `suffix`
- `provider`
- `data env`

1.1 Login del provider y health check

El provider obtiene un JWT de Keycloak (realm `demo`, cliente `dataspace-users`) y lo usa para llamar al Management API. La segunda llamada confirma que `/management/v3/assets/request` responde antes de empezar a crear entidades.

```
export PROVIDER_JWT=$(curl -s -X POST \
  "${KEYCLOAK_URL}/realms/${DS_NAME}/protocol/openid-connect/token" \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d "grant_type=password" \
  -d "client_id=${KC_CLIENT}" \
  -d "username=${PROVIDER_USER}" \
  -d "password=${PROVIDER_PASSWORD}" \
  -d "scope=openid profile email" | jq -r .access_token)

echo "PROVIDER_JWT: ${PROVIDER_JWT:0:40}..."
```

```
curl -s -X POST "${PROVIDER_BASE}/management/v3/assets/request" \
  -H "Authorization: Bearer ${PROVIDER_JWT}" \
  -H "Content-Type: application/json" \
  -d '{"@context":{"@vocab":"https://w3id.org/edc/v0.0.1/ns/"},"offset":0,"limit":1,"filterExpression":[]}' \
  | jq .
```

1.2 Contadores iniciales

Fotografía del estado del provider antes de empezar a crear entidades. Los mismos contadores se usarán al final de la Fase 3 para auditar lo que ha cambiado.

```
for type in asset policyDefinition contractDefinition contractAgreement transferProcess vocabulary; do
  count=$(curl -s -X POST "${PROVIDER_BASE}${ENDPOINTS[$type]}" \
    -H "Authorization: Bearer ${PROVIDER_JWT}" \
    -H "Content-Type: application/json" \
    -d '{"@context":{"@vocab":"https://w3id.org/edc/v0.0.1/ns/"},"filterExpression":[]}' \
    | jq 'length')
```

```
echo "${type}: ${count}"
done
```

El resultado esperado es un resumen de lo que hay dentro del dataspace a nivel de assets, políticos, contratos, agreements y transferencias.

```
asset: 0
policyDefinition: 1
contractDefinition: 0
contractAgreement: 0
transferProcess: 0
```

1.3 Crear vocabulario (categoría **dataset**)

Un vocabulario **dataset** define el esquema JSON que valida los metadatos del asset. Se crea antes que la política y el asset porque el asset puede declarar **assetType: dataset** referenciando esta categoría.

```
export VOCAB_ID="vocab-`${SUFFIX}`"

curl -s -w "\nHTTP %{http_code}\n" -X POST "${PROVIDER_BASE}/management/vocabularies" \
  -H "Authorization: Bearer ${PROVIDER_JWT}" \
  -H "Content-Type: application/json" \
  -d "$(cat <<EOF
{
  "@context": {"@vocab": "https://w3id.org/edc/v0.0.1/ns/"},
  "@id": "${VOCAB_ID}",
  "name": "Taller dataset vocabulary ${SUFFIX}",
  "category": "dataset",
  "connectorId": "${PROVIDER}",
  "jsonSchema": "{\"title\":\"Dataset metadata vocabulary\", \"type\":\"object\", \"properties\": {\"name\": {\"type\":\"string\"}, \"description\": {\"type\":\"string\"}}, \"required\": [\"name\"]}"
}
```

```
EOF
)" | jq .
```

La parte del medio es lo que define los metadatos del vocabulario y luego el apartado `jsonSchema` define el vocabulario en sí.

Según el OpenAPI spec:

- `POST /vocabularies` = **Crear** un vocabulario nuevo (devuelve `IdResponse`)
- `PUT /vocabularies` = **Actualizar** un vocabulario existente (devuelve 204, y si no existe devuelve 404)

Verificación de que el vocabulario aparece listado:

```
curl -s -X POST "${PROVIDER_BASE}/management/vocabularies/request" \
  -H "Authorization: Bearer ${PROVIDER_JWT}" \
  -H "Content-Type: application/json" -d '{}' \
  | jq '.[] | select(.@id=="'${VOCAB_ID}')"'
```

1.4 Crear Access Policy (`PURPOSE = research`)

La Access Policy controla **quién** puede ver la oferta en catálogo. Aquí sólo verá la oferta quien declare la finalidad `research` .

```
export ACCESS_POLICY_ID="access-${SUFFIX}"

curl -s -X POST "${PROVIDER_BASE}/management/v3/policydefinitions" \
  -H "Authorization: Bearer ${PROVIDER_JWT}" \
  -H "Content-Type: application/json" \
  -d "$(cat <<EOF
{
  "@context": { "@vocab": "https://w3id.org/edc/v0.0.1/ns/", "odrl": "http://www.w3.org/ns/odrl/2/" },
  "@id": "${ACCESS_POLICY_ID}",
  "policy": {
    "@context": "http://www.w3.org/ns/odrl.jsonld",
    "@type": "Set",
    "permission": [
```

```

    {
      "action": "USE",
      "constraint": [
        { "left0perand": "PURPOSE", "operator": "eq", "ri
ght0perand": "research" }
      ]
    }
  ],
  "prohibition": [],
  "obligation": []
}
EOF
)" | jq .

```

Automáticamente se crea la política de uso. El bloque `permission` es la **regla de acceso** de la policy, en lenguaje ODRL. Traducido:

- `@type: "Set"`: define un conjunto de reglas.
- `permission`: lista de acciones permitidas.
- `action: "USE"`: se permite **usar** el recurso.
- `constraint`: condición obligatoria para que aplique ese permiso.
- `left0perand: "PURPOSE"`, `operator: "eq"`, `right0perand: "research"`: solo se permite el uso si la finalidad declarada es `research`.

En corto: **"se puede usar el asset únicamente con propósito de investigación"**.



IMPORTANTE: Aquí es necesario diferenciar entre las políticas de EDC y las que permite crear la interfaz. En EDC existen otros tipos de políticas que sí pueden definirse por API.

1.5 Crear Contract Policy (permiso `USE`)

La Contract Policy fija las **reglas de uso** que asume el consumer cuando firma el contrato. En este taller permitimos solo `USE` sin obligaciones.

```
export CONTRACT_POLICY_ID="contract-`${SUFFIX}`"

curl -s -X POST "${PROVIDER_BASE}/management/v3/policydefinitions" \
  -H "Authorization: Bearer ${PROVIDER_JWT}" \
  -H "Content-Type: application/json" \
  -d "$(cat <<EOF
{
  "@context": { "@vocab": "https://w3id.org/edc/v0.0.1/ns/", "odrl": "http://www.w3.org/ns/odrl/2/" },
  "@id": "${CONTRACT_POLICY_ID}",
  "policy": {
    "@context": "http://www.w3.org/ns/odrl.jsonld",
    "@type": "Set",
    "permission": [ { "action": "USE" } ],
    "prohibition": [],
    "obligation": []
  }
}
EOF
)" | jq .
```



AVISO: Por qué `obligation: []` y no `NOTIFY`

El motor de validación de políticas del conector EDC del taller **no soporta** `obligation` en la Contract Policy. Si se añade `obligation: [ { "action": "NOTIFY" } ]`, la oferta se publica en catálogo pero la negociación de Fase 2 termina en `state=TERMINATED` con el error del provider:

```
"Policy in the contract agreement is not equal to the one in the contract offer".
```

Por eso la Contract Policy del taller solo lleva `permission: USE` y deja vacíos `prohibition` y `obligation`.

Listado para confirmar que ambas políticas existen:

```
curl -s -X POST "${PROVIDER_BASE}/management/v3/policydefinitions/request" \
  -H "Authorization: Bearer ${PROVIDER_JWT}" \
  -H "Content-Type: application/json" \
  -d '{"@context":{"@vocab":"https://w3id.org/edc/v0.0.1/ns/"},"offset":0,"limit":50}' \
  | jq '.[] | {id: ."@id", type: .policy."@type"}'
```

1.6 Crear Asset tipo **dataset** (referencia HTTP)

El asset es lo que realmente se ofrece. Para el taller se usa un `HttpData` público (jsonplaceholder.typicode.com/todos), que evita tener que cargar un fichero a MinIO.

```
export ASSET_ID="asset-${SUFFIX}"

curl -s -X POST "${PROVIDER_BASE}/management/v3/assets" \
  -H "Authorization: Bearer ${PROVIDER_JWT}" \
  -H "Content-Type: application/json" \
  -d "$(cat <<EOF
{
  "@context": {
    "@vocab": "https://w3id.org/edc/v0.0.1/ns/",
    "dct": "http://purl.org/dc/terms/",
    "dcat": "http://www.w3.org/ns/dcat#"
  },
  "@id": "${ASSET_ID}",
  "@type": "Asset",
  "properties": {
    "name": "Asset taller ${SUFFIX}",
    "version": "1.0.0",
    "shortDescription": "Asset creado en directo en el taller",
    "assetType": "dataset",
    "dct:description": "Dataset de prueba para flujo end-to-end",
    "dcat:keyword": ["taller", "demo", "dataspace"]
  },
}
```

```

    "dataAddress": {
      "type": "HttpData",
      "baseUrl": "https://jsonplaceholder.typicode.com/todos",
      "name": "todos"
    }
  }
EOF
)" | jq .

```

Lectura por id para confirmar persistencia:

```

curl -s "${PROVIDER_BASE}/management/v3/assets/${ASSET_ID}" \
  -H "Authorization: Bearer ${PROVIDER_JWT}" | jq .

```

En `data_address` se define el almacenamiento. En este campo se prueba HttpData con un asset público en otra ubicación. Las páginas del asset son:

- <https://jsonplaceholder.typicode.com>
- <https://jsonplaceholder.typicode.com/todos>

1.7 Crear Contract Definition (asset + access policy + contract policy)

Es la tarjeta que une un asset con sus dos políticas: la que decide quién puede ver la oferta y la que define qué se puede hacer con el dato.

```

export CD_ID="cd-${SUFFIX}"

curl -s -X POST "${PROVIDER_BASE}/management/v3/contractdefinitions" \
  -H "Authorization: Bearer ${PROVIDER_JWT}" \
  -H "Content-Type: application/json" \
  -d "$(cat <<EOF
{
  "@context": { "@vocab": "https://w3id.org/edc/v0.0.1/ns/"
},
  "@id": "${CD_ID}",

```

```

"accessPolicyId": "${ACCESS_POLICY_ID}",
"contractPolicyId": "${CONTRACT_POLICY_ID}",
"assetsSelector": [
  {
    "operandLeft": "https://w3id.org/edc/v0.0.1/ns/id",
    "operator": "=",
    "operandRight": "${ASSET_ID}"
  }
]
}
EOF
)" | jq .

```

Listado de contract definitions resumido:

```

curl -s -X POST "${PROVIDER_BASE}/management/v3/contractdef
initions/request" \
  -H "Authorization: Bearer ${PROVIDER_JWT}" \
  -H "Content-Type: application/json" \
  -d '{"@context":{"@vocab":"https://w3id.org/edc/v0.0.1/n
s/"},"offset":0,"limit":50,"filterExpression":[]}' \
  | jq '.[] | {id: ."@id", access: .accessPolicyId, contrac
t: .contractPolicyId}'

```

1.8 Verificar publicación local (catálogo del provider contra sí mismo)

El mismo provider hace una llamada a DSP (DataSpace protocol), a su propio endpoint interno para confirmar que la oferta queda visible en catálogo. Es la última comprobación de la Fase 1 antes de cambiar al consumer.

```

curl -s -X POST "${PROVIDER_BASE}/management/v3/catalog/req
uest" \
  -H "Authorization: Bearer ${PROVIDER_JWT}" \
  -H "Content-Type: application/json" \
  -d "${cat <<EOF
{
  "@context": { "@vocab": "https://w3id.org/edc/v0.0.1/ns/"

```

```

},
"@type": "CatalogRequest",
"counterPartyAddress": "${PROVIDER_PROTOCOL}",
"counterPartyId": "${PROVIDER}",
"protocol": "dataspace-protocol-http"
}
EOF
)" | jq .

```

Como resultado se ve el resumen del catálogo. Dentro de este aparece el asset recién creado. Esta respuesta es el **catálogo DSP** que devuelve el provider al ejecutarse una consulta contra sí mismo (paso 1.8). Resumen por bloques:

- `@type: "dcat:Catalog"` : es un catálogo DCAT (lista de ofertas publicadas).
- `dspace:participantId: "conn-citycouncil-demo"` : quién publica ese catálogo (el provider).
- `dcat:dataset` : el asset publicado.
 - `@id: "asset-"` : ID del asset.
 - `assetType: "dataset"` , `name` , `description` , `keywords` : metadatos del recurso.
 - `odrl:hasPolicy` : la oferta contractual asociada al asset.
- `odrl:hasPolicy` :
 - `@id` : identificador de la **offer policy** generada para catálogo.
 - `@type: "odrl:Offer"` : política ofertada al consumidor.
 - `permission (USE)` , `prohibition: []` , `obligation: []` : coincide exactamente con la Contract Policy del paso 1.5.
- `dcat:distribution` : formatos de transferencia que admite ese asset:
 - `HttpData-PULL`
 - `HttpData-PUSH`
 - `AmazonS3-PUSH`

Cada uno apunta al `endpointUrl` del protocolo DSP.
- `dcat:service` : servicio de acceso (endpoint del conector provider en `/protocol`).
- `@context` : prefijos JSON-LD (dcat, dct, odrl, dspace, edc).

Lo más relevante para seguir el taller:

- El asset **sí está publicado** en catálogo.
- Ya queda disponible la `odrl:hasPolicy.@id` que luego necesita el consumer para negociar.
- El provider anuncia varios `transferType` posibles para fase 3.



IMPORTANTE: `DSP` significa **Dataspace Protocol**.

En contexto del taller:

- Es el protocolo estándar que usan provider y consumer para hablar entre conectores.
- Cubre flujos como:
 - consulta de catálogo,
 - negociación de contrato,
 - transferencia de datos.
- Cuando aparece `protocol: "dataspace-protocol-http"`, es la implementación HTTP de DSP.

En corto: **DSP es el “idioma común” entre conectores de un data space.**

`DCAT` significa **Data Catalog Vocabulary** (estándar W3C).

En el taller:

- Es el vocabulario semántico para describir catálogos y datasets.
- Por eso aparecen claves como:
 - `dcat:Catalog`
 - `dcat:dataset`
 - `dcat:distribution`
 - `dcat:DataService`

1.9 Resumen de IDs generados en la Fase 1

Referencia para la Fase 2: estos son los IDs que tiene el provider después de la ronda de creación. La política de oferta que aparece para el consumer en el

catálogo es generada por el conector y se captura en la Fase 2 como

`OFFER_POLICY_ID` .

```
cat <<EOF

===== FASE 1 - PROVIDER OK =====
SUFFIX:                ${SUFFIX}
VOCAB_ID:              ${VOCAB_ID}
ACCESS_POLICY_ID:      ${ACCESS_POLICY_ID}
CONTRACT_POLICY_ID:    ${CONTRACT_POLICY_ID}
ASSET_ID:              ${ASSET_ID}
CD_ID:                 ${CD_ID}
=====

EOF
```

```
{
  printf 'export SUFFIX=%q\n' "$SUFFIX"
  printf 'export VOCAB_ID=%q\n' "$VOCAB_ID"
  printf 'export ACCESS_POLICY_ID=%q\n' "$ACCESS_POLICY_ID"
  printf 'export CONTRACT_POLICY_ID=%q\n' "$CONTRACT_POLICY_ID"
  printf 'export ASSET_ID=%q\n' "$ASSET_ID"
  printf 'export CD_ID=%q\n' "$CD_ID"
} > "/Users/jpardo/Desktop/Proyectos/INESdata/data_spaces/demos/API/phase1_env.sh"
```